

Hash Table and Heap

Author: Eric Zhou

Email: eric_ds_zhou@outlook.com

Date: Oct 5, 2026

Introduction

This technical note summarizes the tools and logic used in Hash Table and heap.

Table of contents

- What Is a Good Hash Table?
- Type of Hash Tables
 - Chained Hashing
 - Open Address Hashing
- Example: Time Complexity Analysis
- Table Doubling
- What Is a valid heap?
- Heap operation
 - Insert in heap
 - Replace in heap
- Analysizing Alogrithms Using Heap
- Bonus: Leetcode Two Sum

What Is a Good Hash Table

According to the note, Good hash functions should have the following properties:

(1) All hash table locations are equally likely to be accessed.

We should try our best to distribute keys uniformly across all buckets.

Bad Hash Table:

$$h(K) = (9K) \bmod 78$$

Since $\gcd(9, 78) = 3$, the hash function can only produce multiples of 3 (0, 3, 6, ..., 75). Therefore, only 1/3 of the buckets can be used.

(2) Keys with a "regular" pattern should not be mapped to the same locations.

Bad Hash Table:

$$h(K) = K \bmod 10$$

We can easily see that keys 10, 20, and 30 will all be mapped to bucket 0, resulting in multiple collisions.

In addition, the size of the hash table should be greater than or equal to the modulus. Otherwise, some hash values may exceed the valid index range. Typically, we choose the modulus to be equal to the table size.

Table Size n	Modulus m	Result
10	10	Valid
10	7	Valid, but buckets 7–9 are unused
10	13	Invalid: indices 10–12 are possible

Type of Hash Tables

There are two common ways to handle collisions in hash tables: chained hashing and open addressing hashing.

Both are valid approaches, but they use different methods to handle collisions and may have different time complexities for dictionary operations.

Chained Hashing

Chained hashing is a common way to implement a hash table. When a collision occurs, we can store multiple elements in the same bucket, typically using a linked list.

For example, suppose we have a hash table of size 5 with the following hash function:

$$h(K) = K \bmod 5$$

If we insert keys 12, 7, 22, 9, and 14, we can see that keys 12, 7, and 22 are all mapped to bucket 2, while keys 9 and 14 are mapped to bucket 4.

Instead of searching for another empty bucket, chained hashing stores the elements in a chain associated with each bucket.

The following table summarizes the five main operations in chained hashing.

Method	Description
Insert (K , D)	Appends a new key-data pair (K , D) to the linked list at $H[h(K)]$.
Member (K)	Returns true if key K exists in the hash table; otherwise, returns false .
Retrieve (K)	Returns the data associated with key K .
Add (K , x)	Finds key K and adds x to its current data ($e.data = e.data + x$).
Replace (K , D)	Finds key K and replaces its current data with D .

The time complexities of these operations are as follows:

Method	Best Case	Expected Case	Worst Case
Insert (K , D)	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
Member (K)	$\Theta(1)$	$\Theta(1 + n/m)$	$\Theta(n)$
Retrieve (K)	$\Theta(1)$	$\Theta(1 + n/m)$	$\Theta(n)$

Method	Best Case	Expected Case	Worst Case
<code>Add(K, x)</code>	$\Theta(1)$	$\Theta(1 + n/m)$	$\Theta(n)$
<code>Replace(K, D)</code>	$\Theta(1)$	$\Theta(1 + n/m)$	$\Theta(n)$

Where:

- n = Number of elements stored in the hash table.
- m = Number of buckets in the hash table.

The expected time complexity assumes uniform hashing.

Open Address Hashing

Open address hashing is another common way to handle collisions in a hash table.

Unlike chained hashing, open address hashing stores all elements directly in the hash table. When a collision occurs, we search for another available bucket using a **probing sequence**.

For example, suppose we have a hash table of size 7 with the following hash function:

$$h(K, j) = (K + j) \bmod 7$$

where $j = 0, 1, 2, \dots, 6$ is the number of probing attempts.

If we insert keys 10, 17, 24, 8, and 15, collisions will occur because several keys initially map to the same bucket.

Using linear probing, the final hash table will look like this:

Hash Table (size = 7)

Index	Key
0	-
1	8
2	15
3	10
4	17
5	24
6	-

For example, when inserting key 17, bucket 3 is already occupied by key 10. Therefore, we increase j to 1 and try bucket 4.

Similarly, key 24 first tries buckets 3 and 4 before being stored in bucket 5.

Unlike chained hashing, no linked lists are needed because all keys are stored directly in the array.

The following table summarizes the main operations in open address hashing.

Method	Description
<code>Insert(K, D)</code>	Searches for an empty bucket using a probing sequence and stores <code>(K, D)</code> in that bucket.
<code>Member(K)</code>	Checks whether key <code>K</code> exists by following the probing sequence. Returns <code>true</code> or <code>false</code> .
<code>Retrieve(K)</code>	Searches for key <code>K</code> using the probing sequence and returns its associated data.
<code>Delete(K)</code>	Searches for key <code>K</code> and marks its bucket as deleted using <code>flagDeleted = true</code> .
<code>Retrieve(K)</code> (with deletion)	Returns the data only if key <code>K</code> is found and its deletion flag is <code>false</code> .

The time complexities of these operations are as follows:

Method	Best Case	Expected Case	Worst Case
<code>Insert(K, D)</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
<code>Member(K)</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
<code>Retrieve(K)</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
<code>Delete(K)</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
<code>Retrieve(K)</code> (with deletion)	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$

Where:

- n = Number of elements stored in the hash table.
- m = Number of buckets in the hash table.
- $\alpha = n/m$ = Load factor.

Example: Time Complexity Analysis

Consider the following algorithm:

```
function Func(A[], n)
    HashTable.Init()
    x = 0

    for i = 1 to n do
        if HashTable.Member(A[i]) then
            HashTable.Add(A[i], 1)
        else
            HashTable.Insert(A[i], 10)
        end if
    end for
```

```

x = 0

for i = 1 to n do
    if HashTable.Member(A[i]) then
        x = x + 1
    end if
end for

return x
end function

```

Assume that the hash table uses **Open Address Hashing (OAH)** and that its size is at least $2n$.

We want to determine the **best, expected, and worst case running times**.

Best Case:

Each hash table operation takes $\Theta(1)$ time in the best case.

Since both loops execute n times, the total running time is:

$$T_{\text{best}}(n) = n\Theta(1) + n\Theta(1) = \boxed{\Theta(n)}$$

Expected Case:

$$ET(n) = n\Theta(1) + n\Theta(1) = \boxed{\Theta(n)}$$

Worst Case:

In the worst case, **Member** and **Add** may each take $\Theta(n)$ time due to collisions and repeated probing.

The first loop may perform both **Member** and **Add** in each iteration, while the second loop performs **Member**.

Thus, using the classroom worst-case bounds:

$$T_{\text{worst}}(n) = n(\Theta(n) + \Theta(n)) + n\Theta(n) = \boxed{\Theta(n^2)}$$

Summary:

Case	Time Complexity
Best Case	$\Theta(n)$
Expected Case	$\Theta(n)$
Worst Case	$\Theta(n^2)$

Table Doubling

What Is a valid heap?

Heap operation

Insert in heap

Replace in heap

Analysizing Algorithms Using Heap

Bonus: Leetcode Two Sum

Since we learned Two Sum with hash tables, why don't we take a look at the actual LeetCode Two Sum problem?

Question:

You are given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Intuition

It's clear that for each element `A[i]`, we need to find its pair, `target - A[i]`.

The easiest way to solve this is to use two `for` loops to check all possible pairs in the array. However, the time complexity of this approach is $\Theta(n^2)$.

Therefore, we can use a hash table to improve the efficiency.

Approach

Since a well-implemented hash table supports dictionary operations such as `member`, `retrieve`, `add`, and `delete` in expected $\Theta(1)$ time, we can use a hash table to store the numbers we have already seen.

In Java, we use `containsKey()`, `get()`, and `put()` to perform these dictionary operations.

For each `nums[i]`, we first calculate:

$$need = target - nums[i]$$

Then, we check whether `need` is already in the hash table.

- If it is, we use `get()` to retrieve its index and return the two indices.
- Otherwise, we use `put()` to store `nums[i]` as the key and `i` as its value.

Time complexity: $\Theta(n)$

Code

```
class Solution {  
    public int[] twoSum(int[] nums, int target) {
```

```
HashMap<Integer, Integer> map = new HashMap<>();

for (int i = 0; i < nums.length; i++) {
    int need = target - nums[i];

    if (map.containsKey(need)) {
        return new int[]{map.get(need), i};
    }

    map.put(nums[i], i);
}

return new int[]{};
}
```

Reference List

1. "CSE2331_Midterm_3_Review" Created by Professor Painter.
2. "CSE2331_Heaps_Homework" Created by Professor Painter.
3. "CSE2331_Hashing_Homework" Created by Professor Painter.
4. LeetCode. "1. Two Sum." <https://leetcode.com/problems/two-sum/>